

Local Postgres with Docker + pgAdmin → Supabase

A complete, end-to-end workflow for practice datasets, dumps, and restores

Learning goals for today (60 minutes)

- Explain Docker (images, containers, volumes, networks) in plain language
- Bring up Postgres + pgAdmin with one docker compose command
- Connect two ways: terminal (psql) and GUI (pgAdmin)
- Understand what you saw: database vs schema vs table
- Load the pgExercises practice dataset (without creating a “cd” schema, if desired)
- Dump and restore data locally, then load the same dataset into Supabase

Plan and timing

- 0–10: Docker concepts + install/verify
- 10–25: docker compose (read every line) + bring up services
- 25–35: Connect via psql + pgAdmin (and debug common failures)
- 35–45: Import pgExercises dataset + verify tables + run queries
- 45–55: pg_dump / pg_restore (what they are and why we care)
- 55–60: Supabase: connect + load + what changes in managed Postgres

Part 1

Docker fundamentals (what it is, why we use it)

Why Docker for databases in a class?

- Consistency: everyone runs the same Postgres version with the same settings
- Isolation: your system Postgres (if any) doesn't conflict with class work
- Repeatability: “delete + recreate” becomes safe and normal
- Portability: a compose file is documentation you can execute
- Real-world skill: containerized services are standard in modern dev workflows

Docker vocabulary (the minimum you need)

- Image = a read-only template (like a “frozen install” of software)
- Container = a running instance of an image (a process with isolation)
- Volume = durable storage that survives container deletion
- Network = a virtual LAN for containers (service names become DNS)
- Port mapping = how your host talks to a container (e.g., localhost:5432 → db:5432)

Install & verify (Docker Desktop + command line)

```
# 1) Install Docker Desktop (Mac/Windows)
```

```
# 2) Then verify:
```

```
docker version
```

```
docker compose version
```

```
# Optional: see what is running
```

```
docker ps
```

Part 2

docker compose: read it like a contract

Our docker-compose.yml (full file)

```
version: "3.9"

services:
  db:
    image: postgres:16
    container_name: cst-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d app"]
```

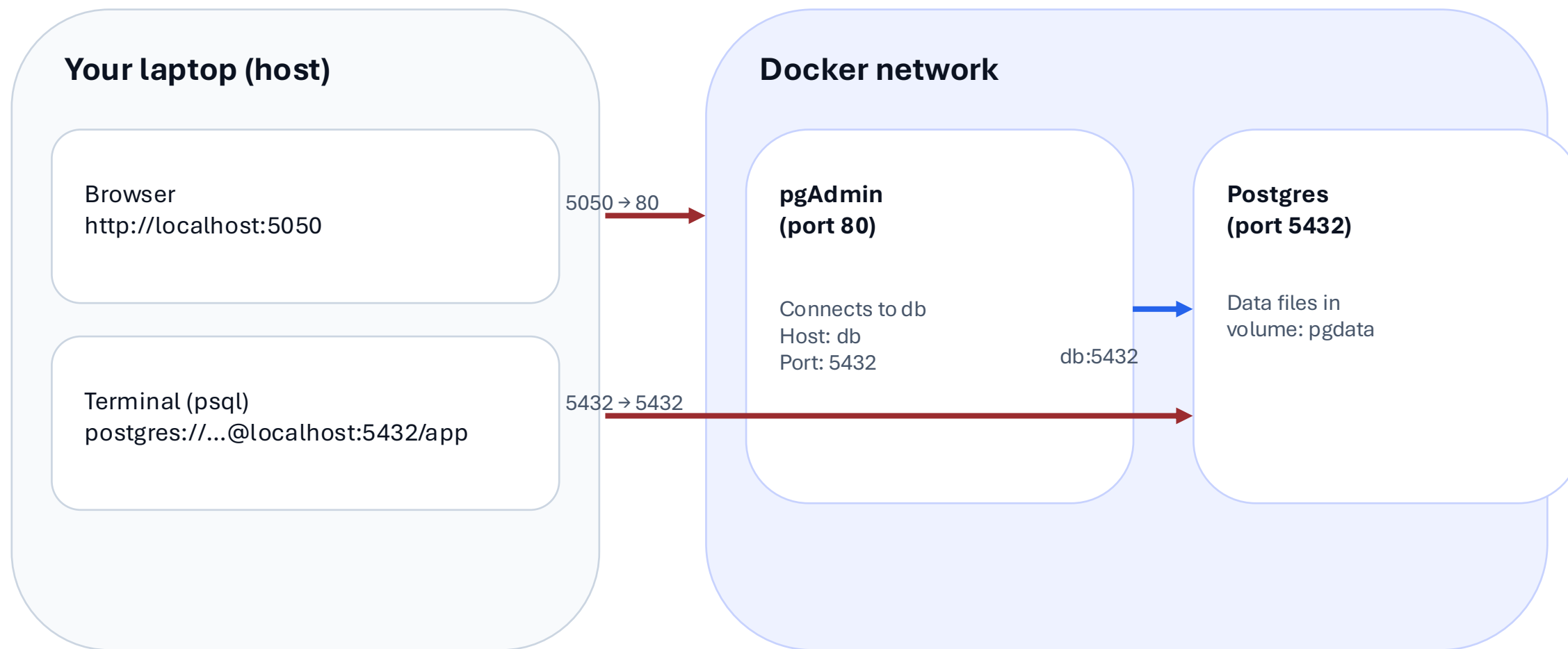
Note: inside Docker, pgAdmin will connect to host “db” (service name).

From your laptop, you connect to Postgres via localhost:5432.

Compose file: what each part is doing

- POSTGRES_DB creates an initial database (here: app)
- POSTGRES_USER/PASSWORD create a role and set a password
- pgdata volume stores your actual database files
- pgadmin-data volume stores pgAdmin config + saved server connections
- healthcheck + depends_on prevent “pgAdmin starts before Postgres is ready”

How the pieces talk to each other



Bring everything up (and confirm it is healthy)

```
# Start services in the background
docker compose up -d

# Check status
docker compose ps

# View logs (useful when something fails)
docker logs -f cst-postgres
```

If you ever need a reset: `docker compose down -v` (WARNING: deletes volumes/data)

Healthy checks are your friend; they turn “mystery failures” into visible states.

Part 3

Connect to Postgres two ways

Connect from your terminal (host machine)

```
# If you have psql installed:  
psql "postgresql://postgres:postgres@localhost:5432/app"  
  
-- In psql:  
conninfo  
l  
dn  
dt  
SELECT version();
```

Connect from inside the Postgres container

```
# Open an interactive shell in the container  
docker exec -it cst-postgres bash
```

```
# Inside container:  
psql -U postgres -d app
```

```
-- In psql:  
conninfo
```

Why you only saw databases “postgres” and “app”

- Postgres always has default/system databases (including postgres)
- Our compose file created one “application” database named app
- A “database” contains schemas; schemas contain tables, views, functions
- So: if you expected “cd,” that was a schema name from the pgExercises dataset, not a database

Open pgAdmin in your browser

```
# Open this in your browser:  
http://localhost:5050
```

```
Login:  
Email: admin@cst.local  
Password: admin
```

pgAdmin: add the Postgres server (the common mistake)

- Register → Server
- Name: Local Docker Postgres
- Connection tab:
- Host name/address: db (NOT localhost)
- Port: 5432
- Maintenance DB: app
- Username: postgres
- Password: postgres (save password)

Part 4

Database vs schema, then import pgExercises

Database vs schema (a clean mental model)

DATABASE

- A database is a logical container within a Postgres server
- Connections choose a database (.../app)
- Databases separate ownership, extensions, and some settings
- You cannot query across databases directly (without dblink/FDW)

SCHEMA

- A schema is a namespace inside a database
- Schemas organize objects: tables, views, functions
- Two schemas can both have a table named members
- `search_path` decides which schema is used by default

About pgExercises “Club Database”

- A small relational dataset designed for SQL practice
- Tables include: facilities, members, bookings
- Great for joins, GROUP BY, date filtering, subqueries
- We will import it so you can practice locally and in Supabase

Verify tables and run first sanity queries

```
-- In psql or pgAdmin Query Tool:  
dt  
  
SELECT count(*) FROM members;  
SELECT count(*) FROM facilities;  
SELECT count(*) FROM bookings;  
  
-- Sample join:  
SELECT m.firstname, m.surname, b.starttime  
FROM bookings b  
JOIN members m ON b.memid = m.memid  
ORDER BY b.starttime DESC  
LIMIT 10;
```

Mini practice set (5 queries to do together)

- List the names of all facilities (order by name)
- Show the total number of bookings
- Find bookings in September 2012
- Compute revenue per facility (membercost/guestcost × slots)
- Find members who have never made a booking

Part 5

Dump + restore (portable Postgres backups)

Dump the database (two common formats)

```
# Plain SQL (human-readable)
pg_dump "postgresql://postgres:postgres@localhost:5432/app" > app_dump.sql

# Custom format (smaller + flexible restore)
pg_dump -Fc "postgresql://postgres:postgres@localhost:5432/app" > app_dump.dump
```

Plain SQL restores with: `psql -f app_dump.sql`

Custom format restores with: `pg_restore` (more control)

Dump from inside Docker (no host tools needed)

```
# Stream a dump to your host filesystem
```

```
docker exec -t cst-postgres pg_dump -U postgres -d app > app_dump.sql
```

```
# Custom format
```

```
docker exec -t cst-postgres pg_dump -U postgres -d app -Fc > app_dump.dump
```

Restore into a fresh database (practice “recreate it”)

```
# Create a fresh DB
```

```
createdb "postgresql://postgres:postgres@localhost:5432/postgres" app_restore
```

```
# Restore from SQL dump
```

```
psql "postgresql://postgres:postgres@localhost:5432/app_restore" -f app_dump.sql
```

```
# Or restore from custom dump
```

```
pg_restore -d "postgresql://postgres:postgres@localhost:5432/app_restore" app_dump.dump
```

Part 6

Supabase: managed Postgres with modern app tooling

What Supabase is (in one slide)

- Managed Postgres database (you still get SQL, schemas, indexes, roles)
- Adds application services: Auth, Storage, Edge functions, Realtime
- Strong security defaults: Row Level Security (RLS) is central
- Gives you connection strings for psql, drivers, and pooling

Supabase setup checklist (what you click)

- Create a new project (choose region, set a strong DB password)
- Open Project Settings → Database to find connection info
- Copy a connection string for psql (direct or pooler)
- If using IP restrictions, allow your current IP
- Use the SQL Editor for quick imports (small files), or psql for larger imports

Connect to Supabase with psql (direct connection)

```
# Example (you will replace with YOUR project values)
psql "postgresql://postgres:<PASSWORD>@db.<PROJECT_REF>.supabase.co:5432/postgres"

-- Verify
SELECT version();
dn
```

Import the same dataset to Supabase

- Best: `psql -f clubdata_public.sql` against your Supabase connection string
- Alternative: SQL Editor (paste file contents in chunks if needed)
- After import: `dt` and `count(*)` checks again
- Now you can practice pgExercises queries anywhere (no Docker required)

What changes in managed Postgres (important realism)

- You are not a full superuser (some system operations are restricted)
- You may have additional schemas (auth, storage) maintained by the platform
- Connection pooling may be recommended/required for some app patterns
- Security defaults are stronger (RLS is encouraged)
- Backups are platform-managed, but YOU still own logical exports for portability

In-class lab

Hands-on checklist + deliverable

Lab Part A–C (setup + connect + import)

A) Bring up services (check off)

- Run `docker compose up -d`
- Show `docker compose ps` with both services Up
- Open pgAdmin at `localhost:5050`
- Add server connection using `host=db`
- Run `SELECT 1` in pgAdmin Query Tool

B/C) Import dataset (check off)

- Import `clubdata_public.sql` into app
- Verify tables exist in public schema
- Record row counts for members/facilities/bookings
- Run one JOIN query and save results

Lab Part D–F (queries + dump/restore + Supabase)

D) Practice queries (submit SQL)

- Write 5 queries from the mini practice list
- Each query must include a comment describing intent
- Validate at least one result (sanity check)
- Save all SQL in a single .sql file

E/F) Portability (submit proof)

- Create app_dump.sql using pg_dump
- Restore into app_restore and verify a count(*)
- Connect to Supabase and import the same dataset
- Run one query in Supabase and screenshot output

Lab extensions (if you finish early)

- Add an index and compare EXPLAIN plans before/after
- Create a view for “revenue per facility” and query it
- Create a read-only role and grant SELECT on all tables
- Write a short reflection: what broke, how you debugged it, what you learned

Troubleshooting

The top failures and how to diagnose them

Common issue: port already in use (5432 / 5050)

- Symptom: “bind: address already in use” on docker compose up
- Cause: local Postgres or another service is using 5432 (or another pgAdmin uses 5050)
- Fix: stop the conflicting service OR change host port in compose (e.g., 5433:5432)
- Then connect using the new host port (localhost:5433)

Common issue: pgAdmin cannot connect

- If pgAdmin is in Docker: host must be db, not localhost
- If connecting from host tools: host is localhost
- Check credentials match POSTGRES_USER/PASSWORD
- Check db container is healthy (docker compose ps)
- Look at logs: docker logs cst-postgres

Common issue: “I edited compose but password didn’t change”

- Environment vars only apply on first initialization
- If your pgdata volume already exists, Postgres keeps the old credentials
- Fix A: change password using SQL (ALTER USER ...)
- Fix B: reset everything by deleting volumes (docker compose down -v)

Wrap-up

What you now have (and what's next)

You can now...

- Stand up Postgres reliably in a repeatable environment
- Explain and edit a docker compose file with intention
- Connect and debug connectivity (host vs container networking)
- Import a dataset and verify it with sanity checks
- Create portable exports with pg_dump and restore them safely
- Use the same SQL workflow on Supabase (managed Postgres)

Quick reference checklist

- 1) `docker compose up -d`
- 2) `docker compose ps`
- 3) `psql 'postgresql://postgres:postgres@localhost:5432/app'`
- 4) pgAdmin: host=db, port=5432
- 5) Import: `psql ... -f clubdata_public.sql`
- 6) Verify: `\dt + count(*)`
- 7) Dump: `docker exec -i cst-postgres pg_dump ... > app_dump.sql`
- 8) Restore: `psql .../app_restore -f app_dump.sql`
- 9) Supabase: connect + import + verify